

iOSDC Japan 2026 ・ Regular Talk ・ 40 min

映像変換サーバーなしで iPhone端末内で HLSを生成してライブ配信

Hikaru Sato @SatoHikaruDev

playlist.m3u8

init.mp4

000001.m4s

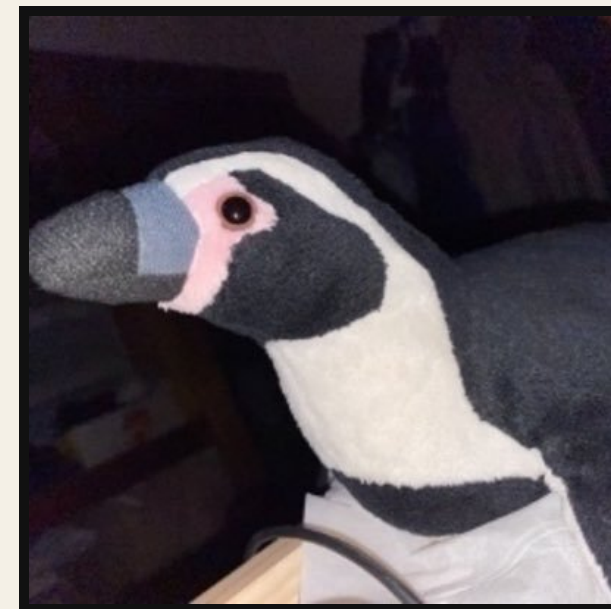
— ABOUT ME

Hikaru Sato

@SatoHikaruDev

主に iOS / Android / Ruby on Rails のアプリ開発をしています

MomentNow という「今この瞬間」の動画をHLSで配信し、
URLで共有できる iOSアプリ を個人開発してます



今回は MomentNow での端末内HLS生成の仕組みや実装を分解します

話すこと

- 01 HLSがライブになる仕組み
- 02 サンプルアプリを動かす
- 03 映像と音声と同じ時間軸へ
- 04 約2秒のfMP4を生成
- 05 保存してからplaylistへ公開
- 06 最後の公開まで完了を管理

01

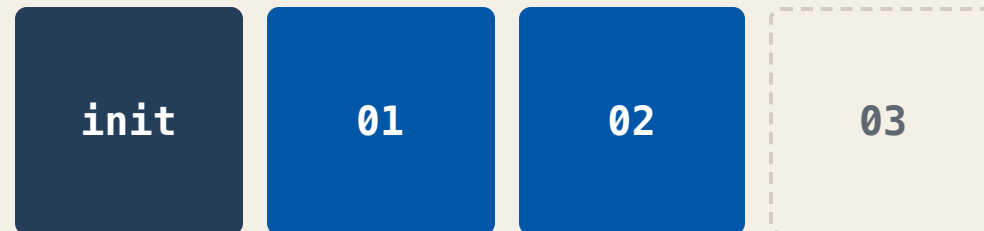
まず、HLSで 「ライブ」になる仕組み

短い動画ファイル + 更新され続けるplaylist

— HLS LIVE IN ONE SENTENCE

HLSは、短い動画と 更新されるplaylistをHTTPで配る

```
#EXTM3U
#EXT-X-MAP: init.mp4
#EXTINF: 2.000
seg/000001.m4s
#EXTINF: 2.000
seg/000002.m4s
```



EXT-X-MAP

最初に読む設定ファイル

EXTINF

次のsegmentの長さ

URI

次に取得するファイル

— THREE FILE TYPES

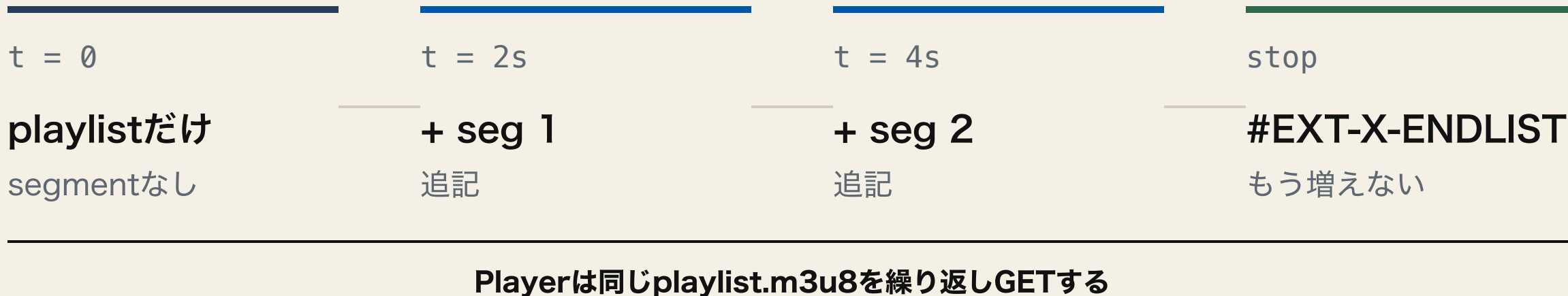
HLSライブ配信を構成する、3種類のファイル

M3U8	MP4	M4S
playlist.m3u8 更新される再生順	init.mp4 再生を始めるための設定	000001.m4s 2秒前後の映像と音声

init.mp4（再生準備） + m4s（約2秒） + playlist（順番） = 最初の再生

— WHY IT IS LIVE

playlistが増えるたび、 Playerは次のsegmentを取得する



— FROM HLS TO THIS TALK

一般的には、サーバーが受信した映像をHLSへ変換する



今回なくすのは、映像を受信し続けて変換するサーバー

1ファイルだと撮影後、共有するためのアップロードに時間がかかる

撮影中からアップロードできていれば共有も楽

撮影終了後

recording.mov

大きな1ファイルを送信



撮影中

init.mp4

001.m4s

002.m4s

小さく区切って順次送信

— THIS TALK'S IMPLEMENTATION

iPhoneが生成したHLSを 撮影中からS3へ公開する

映像と音声を受け取る

Camera + Mic



約2秒に分ける

AVAssetWriter



S3へ保存する

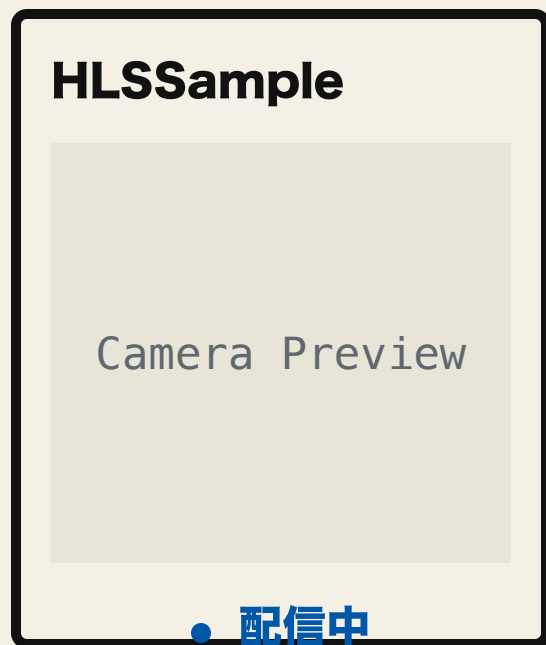
一時PUT URL



再生順を公開する

commit API

仕組みが見えたところで、公開サンプルアプリを動かす



HTTP
PUT

start	+2s	after PUT
PUT init.mp4	PUT seg 1	PUT playlist

Mac HTTP Server / Viewer

LIVE LOCAL

playlist.m3u8を追従再生

github.com/HikaruSato/iosdc2026HLSSample

Server は「PUTされたファイルを残すだけ」

Mac

```
$ python3 server/server.py
```

Viewer: <http://localhost:8080>

iPhone server URL: <http://192.168.x.x:8080>

PUT init.mp4 / m4s / m3u8

DISK server/data/streams/...

GET 同じファイルをViewerへ返す

「配信開始」で、iPhoneがHLS生成を始める

● 配信開始



Camera + Mic

video / audio

CMSampleBuffer



AVAssetWriter

init.mp4 + m4s

elapsed 0.0 → segment 0 →

最初に届くのは、1回だけのinit.mp4

$t \approx 0$

initialization callback



init.mp4

再生を始めるための設定



`PUT /streams/{id}/init.mp4`

media segmentより先に、再生の前提を保存する

以後は約2秒ごとに、2つのPUTが続く

segment 1

PUT 000001.m4s

→

公開

PUT playlist.m3u8

→

segment 2

PUT 000002.m4s

→

公開

PUT playlist.m3u8

playlistが指すのは、保存に成功したsegmentだけ

Viewerは最新streamを見つけ、playlistを追いかける

1 秒ごと

GET /api/streams



最新を選択

playlistURL



LIVE

Safari / AVPlayer / hls.js

STREAM **stream-...** SEGMENTS **1 → 2 → 3** PLAYLIST /streams/.../playlist.m3u8

デモ後に残る、3種類のHLSファイル

```
server/data/streams/stream-20260818-225315-C7A2AB11/
```

└─ init.mp4 1,136 bytes

└─ playlist.m3u8 682 bytes

└─ seg/ 18 files

000001.m4s

000002.m4s

...

000018.m4s

保存先で `ffplay playlist.m3u8` → HLS一式をそのまま再生確認

CameraからViewerまで、4段階で考える

01 取り込む

Camera / Mic →
CMSampleBuffer

→

02 分割する

約2秒分 → init.mp4
/ .m4s

→

03 保存する

ファイル本体 → Mac
/ S3

→

04 公開する

playlistへ追加 →
Viewer

今回の難所は、各境界で「順序」を失わないこと

— ROUTE

カメラの1 frameが、HLSとして視聴者へ届くまで

01

HLSの最小形

02

iOSの責務分割

03

CMSSampleBuffer
と時刻

04

fMP4 segment

05

storage保存と
playlist公開

Public at iOSDC 2026 / iosdc2026HLSSample

— SAME SHAPE, DIFFERENT DESTINATION

同じHLSを、MacまたはS3へ置く

公開サンプル

iPhone

HTTP PUT

Mac filesystem

HTTP GET

Local Viewer

MomentNow

iPhone

presigned
PUT

Amazon S3

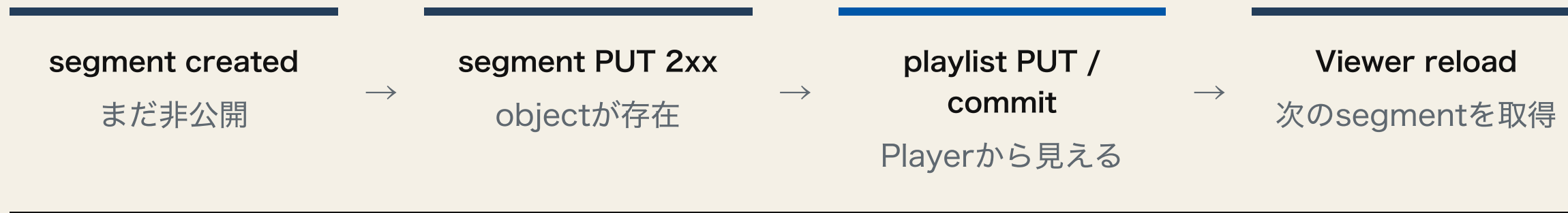
CloudFront

Web Viewer

init.mp4とm4sのバイト列は、どちらもiPhoneが生成する

— CONTROL PLANE

公開タイミングは、playlistが制御する



保存と公開を分ける・playlistはno-cache、init / m4sはlong-cache

fMP4は「設定」と「再生可能な断片」を分ける

1 配信に1つ

init.mp4

Playerが再生を始めるための
設定

+

約2秒ごと

000001.m4s

実際の映像と音声

=

playlistが結ぶ

再生可能

Playerはこの組み合わせを取
得

AVAssetWriterはsegmentを作る。playlistは作らない。

AVFoundation

encode + segment

- H.264 / AAC
- init.mp4
- 約2秒のmedia segment

App

順番を付けて公開

- 再生順番号 (seq)
- S3 upload
- 公開可能なseqを通知

02

動くサンプルアプリを、 4つの責務へ分ける

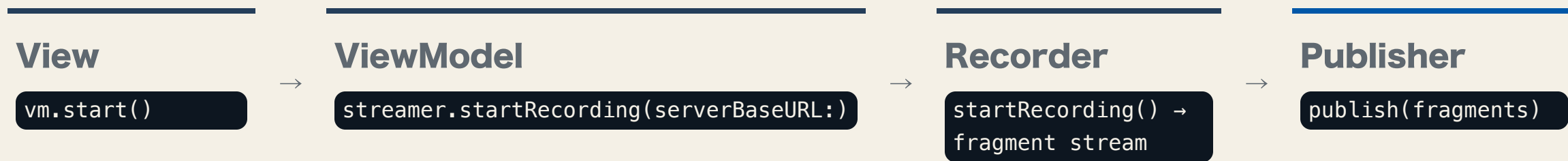
UI / orchestration / media / publication

— FOUR OWNERS

4つの型で生成と公開を分ける

UI	SampleStreamViewModel	idle / streaming / completed
FACADE	SampleHLSStreamer	RecorderとPublisherを接続
GENERATE	HLSegmentRecorder	capture / writer / media clock
PUBLISH	HLSStreamPublisher	HTTP PUT / playlist / retry

配信開始タップで、fragmentの生成と公開をつなぐ



`AsyncThrowingStream<HLSFragment>` ————— `initialization / media`

Queueとactorは、守る状態が違う

MainActor	ViewModelの表示状態	
sessionQueue	AVCaptureSessionの構成・start / stop	
writingQueue	CMSampleBuffer順序・AVAssetWriter状態	
HLSStreamPublisher actor	公開順・retry・playlist状態	

1本の巨大なロックではなく、責任ごとに直列化する

— START ORDER

streamを準備してから、Publisherへ渡す

```
1 let fragments = try await recorder.startRecording()
2
3 let uploadTask = Task {
4     await publisher.publish(fragments)
5 }
6
7 activeStream = ActiveStream(
8     publisher: publisher,
9     uploadTask: uploadTask
10 )
```

03

映像と音声を、 同じ時間軸でWriterへ渡す

Camera / Mic → CMSampleBuffer → one shared timeline

— CAPTURE TOPOLOGY

AVCaptureSessionが、2つのdeviceを2つのDataOutputへつなぐ

Back camera

AVCaptureDeviceInput

Microphone

AVCaptureDeviceInput

AVCaptureSession

VideoDataOutput

CMSampleBuffer

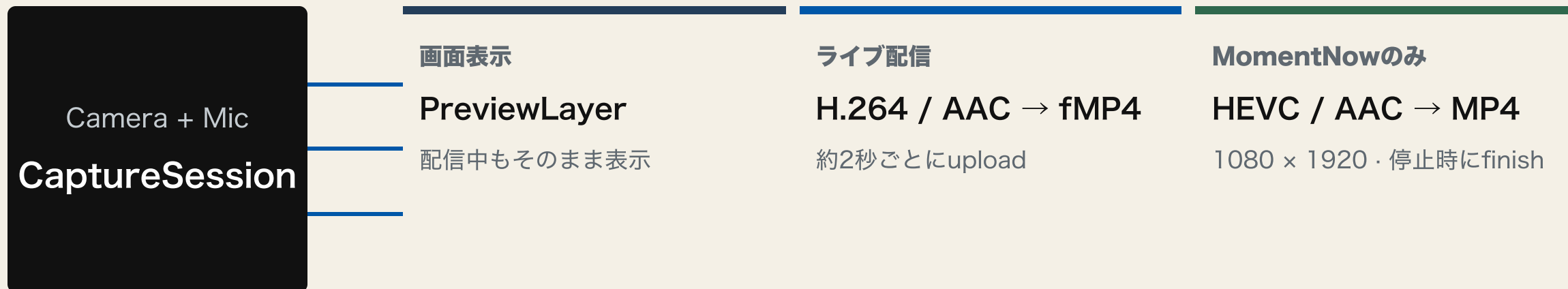
AudioDataOutput

CMSampleBuffer

HLSSegmentRecorder.setupCaptureSessionLocked()

— ONE CAPTURE · THREE OUTPUTS

1つのCaptureSessionから、3つの用途へ分ける



recording.mp4 → 元動画としてupload + 設定時は写真ライブラリへ保存

公開サンプルはPreview + HLS。MomentNowは保存用Writerも並行する

— WHY DATA OUTPUT

完成ファイルではなく、CMSampleBufferを使う

MovieFileOutput

録画ファイルを作る

高レベルで便利



Video / Audio DataOutput

frame / audio blockを受け取る

Writerへ逐次append

video CMSampleBuffer

audio CMSampleBuffer

same writingQueue

— CMSAMPLEBUFFER ≠ SEGMENT

多数のCMSampleBufferを、約2秒のsegmentへまとめる

Camera

video CMSampleBuffer

約1 frame · 30fpsなら約33ms

Microphone

audio CMSampleBuffer

短い音声block



AVAssetWriter

**encode
+ segment**



1 配信に1つ

init.mp4

codec · trackなどの再生設定

約2秒ごと

000001.m4s

video frame + audio block本体

video CMSampleBuffer

カメラから届く約1 frame分

audio CMSampleBuffer

マイクから届く短い音声block

media segment

約2秒分のvideo frame /
audio block

initialization segment

再生開始に必要な設定

CameraのNV12をWriterがH.264に圧縮

Capture

420f / NV12

pixel buffer

append

Writer

H.264 High

encoded media

```
1 videoOutput.videoSettings = [  
2     kCVPixelBufferPixelFormatTypeKey as String:  
3     kCVPixelFormatType_420YpCbCr8BiPlanarFullRange  
4 ]
```

サンプルは1.5 Mbps · 個人アプリは上り回線に合わせて0.9-2.5 Mbps

Writerへ届く前に、2つのguardで落とす

```
captureOutput(_:didOutput:from:)
```

```
isWriting  
== true
```

録画中だけ

```
CMSampleBufferDataIsReady
```

利用可能なbufferだけ

```
startWriterIfNeeded  
→ append
```

```
1 guard isWriting else { return }  
2 guard CMSampleBufferDataIsReady(sampleBuffer) else { return }
```

— SESSION START

最初のvideo frameで、全trackを開始

audio CMSampleBuffer

まだappendしない

first video frame

startWriting

startSession(at: 10s)

video + audio共通

```
1 guard output === videoOutput else { return }
2 guard !didStartSession, let writer else { return }
```

— PTS = CMSAMPLEBUFFER TIMESTAMP

CMSampleBufferの撮影時刻を、Writerの開始時刻へ移す

Capture PTS

CMSampleBuffer

に付く撮影時刻

58342.31

– firstVideoPTS + 10s

Writer

timeline

この配信内の時刻

10.00

10秒起点は今回のWriter設定。HLS仕様の固定値ではない

CMSampleBuffer presentationTimeStamp / writer.initialSegmentStartTime

— ONE DELTA

時刻補正は、全CMSampleBufferへの平行移動

delta = 10s - firstVideoPTS

adjustedPTS = sourcePTS + delta

— A/V SYNC

VideoとAudioを別々に補正しない

video	58342.31	<u>same delta</u>	video	10.00
audio	58342.29		audio	9.98

相対差を保ったまま、Writerのtime rangeへ移す

— APPEND READINESS

Writerへ入らないCMSampleBufferは、待たずに落とす

```
1 if output === videoOutput {  
2     guard videoInput.isReadyForMoreMediaData else { return }  
3     didAppend = videoInput.append(adjustedSampleBuffer)  
4 } else {  
5     guard audioInput.isReadyForMoreMediaData else { return }  
6     didAppend = audioInput.append(adjustedSampleBuffer)  
7 }
```

push source

×

real-time writer

→

bounded latency

04

映像と音声を、 約2秒のfMP4へ分ける

Four HLS settings and a segment delegate

— WHAT UNBLOCKED THE IMPLEMENTATION

macOS向けAppleサンプルが、iOS実装の突破口になった

2025.03

AIで最初の試作

再生できるHLSとして
実用まで至らず



Apple fmp4Writer

正しい生成手順を確認

Writer設定・時刻補正
segment delegate



iPhone

Camera / Micへ置換

AVAssetWriterの核心部分を
iOSで利用

macOSのmovie入力を、iOSのライブCapture入力へ置き換えた

URLなしで、Dataをdelegateから受け取る

```
1 let writer = AVAssetWriter(contentType: .mpeg4Movie)
2 writer.delegate = self
```

AVAssetWriter

MP4 content type

init.mp4

再生準備

media segment

約2秒のData

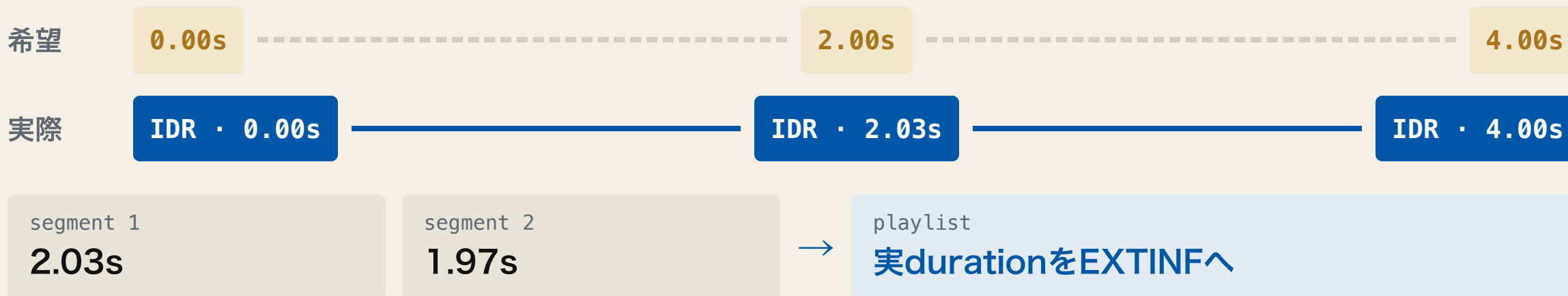
— HLS-SPECIFIC · FOUR KNOBS

HLS出力は、4つのpropertyで有効になる

- 01 `outputFileTypeProfile` Apple HLS向けfMP4
- 02 `preferredOutputSegmentInterval` 希望するsegment間隔
- 03 `initialSegmentStartTime` 最初のsegment開始時刻
- 04 `delegate` 生成されたDataの受け口

— PREFERRED INTERVAL

2秒ぴったりではなく、2秒付近のIDRで切る



IDR ほかのframeを参照せず、そこから単独で再生を始められるkeyframe

2秒は設定値。playlistにはAVAssetSegmentReportの実durationを書く

— KEYFRAME ALIGNMENT

2秒で切るなら、2秒以内にIDRを用意する



```
1 AVVideoMaxKeyFrameIntervalDurationKey: config.segmentSeconds
```

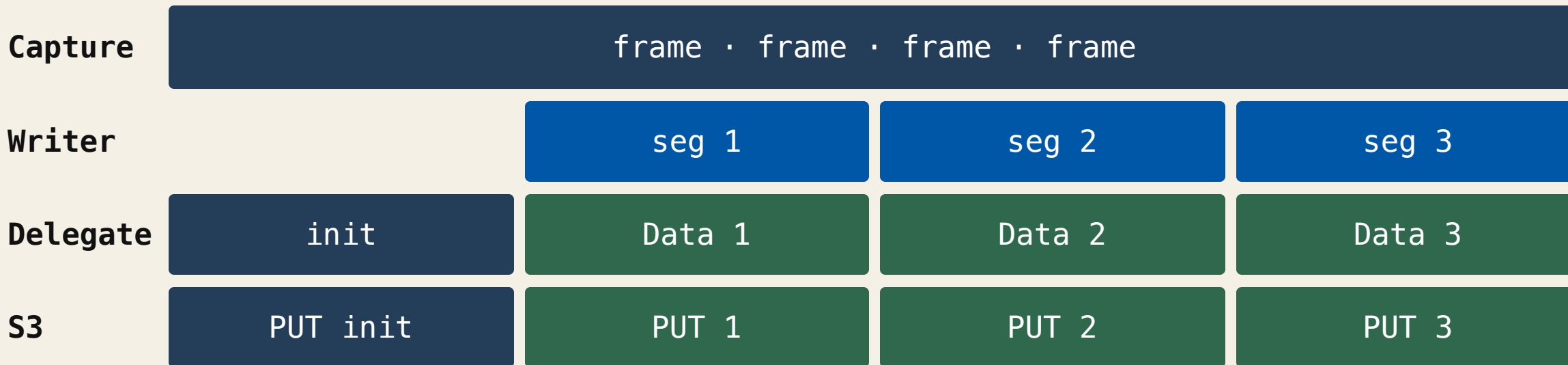
IDR ほかのframeを参照せず、そこから再生を始められるkeyframe

segmentTypeでinitとmediaを分ける

```
1 switch segmentType {
2   case .initialization:
3     onInitSegment?(segmentData)
4   case .separable:
5     segmentIndex += 1
6     onMediaSegment?(segmentIndex, segmentData, segmentReport)
7   @unknown default:
8     break
9 }
```

— GENERATION TIMELINE

Captureを止めず、約2秒ごとにDataを出す



Writerのfinishを待たず、生成とS3 uploadを並行できる

— BACKPRESSURE DECISION

Writerが詰まったら、古いframeを捨てる

Camera	frame 1	frame 2	frame 3	frame 4
Writer	append	append	busy	drop

待って貯める

古い映像が残り、遅延が増える

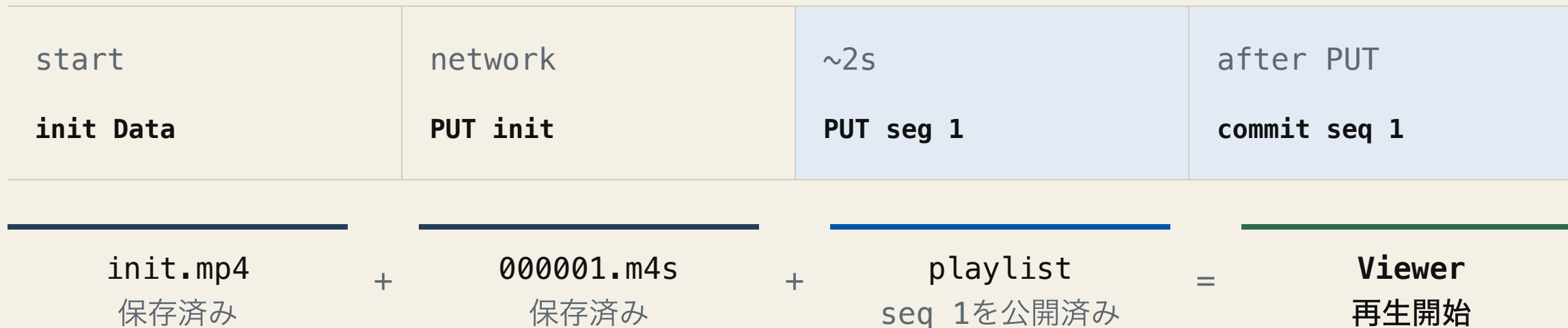
待たずに落とす

少しカクつくが、現在へ追いつく

`isReadyForMoreMediaData == false` は意図的なdrop

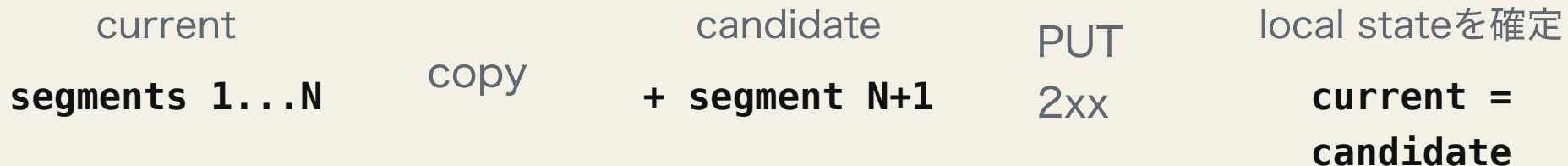
— VISIBLE OUTPUT

再生開始は、init PUTと最初のcommitの後



サンプルは、playlistのPUT成功後に状態を確定

```
1 var nextManifest = manifest
2 nextManifest.addSegment(seq: seq, durationSec: durationSec)
3
4 try await client.putPlaylist(streamId: streamId, text: nextManifest.text)
5 manifest = nextManifest
```



後半のcommit APIとは別。ここではiPhone自身がplaylist本文を書く

— DRAIN BEFORE FINISH

stopは、3つの非同期処理を順番に閉じる

1 Recorder

capture停止 → finishWriting

最後のfragmentが出る

→

2 Stream

continuation.finish()

fragment列を閉じる

→

3 Publisher

uploadTask.value

ENDLIST PUTまで待つ

「撮影停止」と「配信終了」は、同じ瞬間ではない

05

保存できたsegmentだけを、 playlistへ公開する

Save bytes first, then make the segment visible to viewers

「保存先を受け取る → 保存する → 公開する」に分ける

APPの操作	MomentNow	結果
一時保存先を受け取る	POST /presign	短期PUT URL
segment本体を保存する	PUT → S3	objectが存在する
playlistへ載せる	POST /commit	Viewerから見える
playlistを取得する	GET → CloudFront	再生が進む

S3は保存先の選択。HLSで重要なのは「保存してから公開」

— PREPARE BEFORE MEDIA

segment前にstreamIdと再生URLを準備

createStream

streamId / prefix

→

ticket

playbackUrl

→

permission

camera /
microphone

→

ready

previewSession

```
1 if ticket == nil {  
2     ticket = try await apiClient.ticket(streamId: stream.streamId)  
3 }  
4 await requestPermissionsAndSetup()  
5 try await streamer?.start()
```

— INITIALIZATION SEGMENT

init.mp4は、最初に1度だけPUT

```
1 guard !didUploadInit else { return }
2 let ticket = try await api.presign(
3     streamId: stream.streamId, kind: "init", seq: nil
4 )
5 try await api.put(
6     to: URL(string: ticket.putUrl)!, data: initData,
7     contentType: "video/mp4"
8 )
9 didUploadInit = true
```

— MEDIA SEGMENT

mediaごとにpresign → PUT → commit

```
1 let ticket = try await api.presign(  
2     streamId: stream.streamId, kind: HLSUploadKind.segment,  
3     seq: seq  
4 )  
5 try await api.put(  
6     to: URL(string: ticket.putUrl)!, data: segmentData,  
7     contentType: HLSContentType.segmentM4S  
8 )  
9 _ = try await api.commit(  
10     streamId: stream.streamId, seq: seq, durationSec: dur,  
11     isLast: false, targetDurationSec: targetDurationSec  
12 )
```

— NO AWS CREDENTIAL ON DEVICE

iPhoneへ渡すのは、短期のPUT URLだけ

DO NOT

AWS credentialを内包

漏えい範囲が広い



PRESIGNED URL

object単位のPUT権限

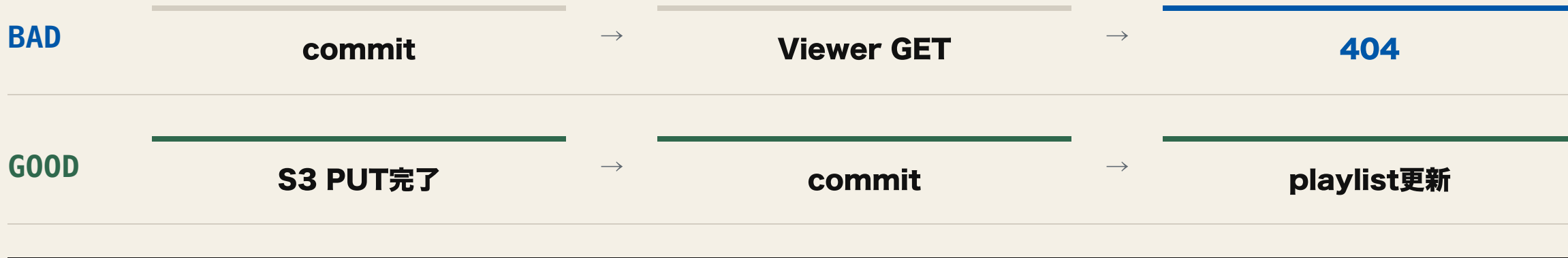
streamId / kind / seqで制約

サーバーが「この1ファイルだけPUTしてよいURL」を一時発行する

```
1 var request = URLRequest(url: presignedURL)
2 request.httpMethod = "PUT"
3 request.setValue(contentType, forHTTPHeaderField: "Content-Type")
4 request.httpBody = data
```

— PUBLISH AFTER BYTES

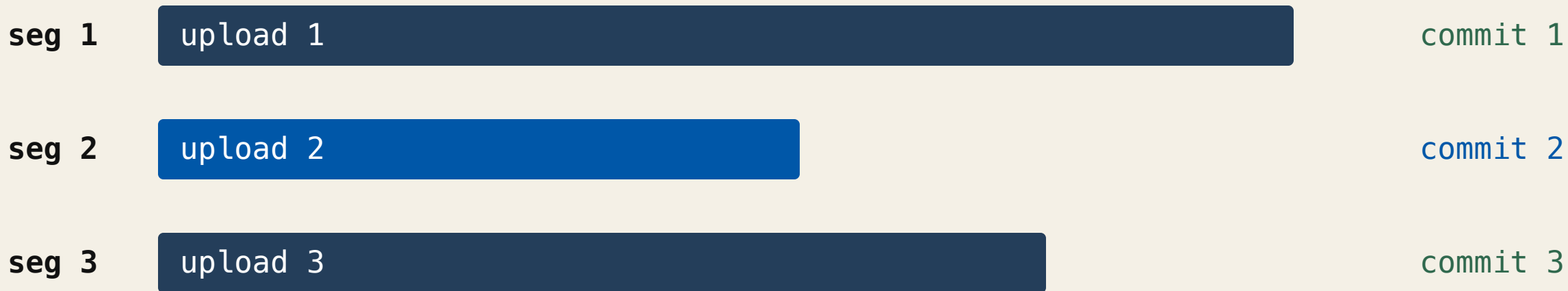
PUT成功より先に、playlistへseqを出さない



サンプルは最大3回retry・失敗したsegmentはplaylistへ載せない

— OUT-OF-ORDER RACE

現在のcommit APIはseq順へ並べ直すが、 欠番は待たない



completion **2, 1, 3** $\neq$ playback **1, 2, 3**

理想は、連続したseqまでだけを公開する「公開済み境界」を持つこと

— ACTOR STATE

Actorは、uploadと終了の状態をまとめて守る

didUploadInit init.mp4の重複PUTを防ぐ

lastCommittedSeq commit成功済みの最大seq

inUploadSegmentCount 停止時に待つbacklog

isFinishing 遅れて完了したsegmentでもENDLISTを再通知

actor内のawaitは再入可能。完了順ではなくseqを正とする

— STOP CONTRACT

現状は、actorへ入った uploadが0になるまで待つ

1

`recorder.stop()`



2

`uploader.finish()`



3

`actor内 pending
== 0`



4

`state =
completed`

```
1 while await streamer.hasPendingUploads() {  
2     try? await Task.sleep(for: .milliseconds(500))  
3 }
```

理想はupload Task／イベント列そのものとfinal commitをawaitすること

06

Writerを待たせず、 最後の公開完了まで管理する

Callback → Task → upload completion

callbackでは待たず、TaskへDataを引き渡す

LiveSegmentRecorder

init Data / media Data

WRITING QUEUE

callbackを返す

次のCMSampleBuffer処理へ

SWIFT TASK

Uploaderをawait

presign / PUT / commit

initialization →
uploadInitIfNeeded

separable → uploadSegment segmentReport → duration改善余地

— PUBLIC DEMO → PRODUCTION

個人アプリでは、HLSの外側に4つの制御を足す

AUTH

user / group
ownership

PRESIGN

object単位の短期PUT
権限

COMMIT

公開順とplaylistの同時
更新を制御

DELIVERY

CloudFront / ticket /
status

AVAssetWriterの後ろを差し替えれば、同じiOS pipelineを使える

— BACK TO THE DEMO

冒頭のデモで増えていたのは
「ファイル」と「再生可能な順序」

init.mp4 + 000001.m4s + playlist更新 = LIVE

— FIT, NOT UNIVERSAL

この構成が効くのは、用途を絞ったとき

GOOD FIT

少人数・短時間・単一品質

新しいiPhone / 安定した上り回線

USE A MEDIA PLATFORM

大規模・長時間・ABR・厳しいSLA

発熱、電池、回線変動を吸収したい

「端末でできる」と「端末でやるべき」は別の判断

— TAKEAWAYS

HLSライブ配信は、4つの順序で成立する

01 時刻 ・ Clock

Capture PTSをWriterのtimelineへ移す

02 区切り ・ Boundary

IDRとsegment intervalをそろえる

03 公開 ・ Upload

DataをS3へPUTしてからcommitする

04 終了 ・ Finish

全送信とfinal commitの終了を待つ

ありがとうございました

APPENDIX

時間が余ったとき／Q&A用

設定値・時刻補正・サーバー運用の詳細

9ファイルで、生成から再生まで追える

HLSSegmentRecorder.swift	capture / encode / segment	SampleHLSStreamer.swift	recorderとuploadを接続
HLSStreamPublisher.swift	順序 / retry / finish	HLSManifest.swift	playlist state
HTTPHLSClient.swift	PUT / health check	SampleStreamViewModel.swift	permission / UI state
ContentView.swift	操作と観測	server.py	atomic replace / GET
static/app.js	latest stream / hls.js		

— CONFIG

上り回線に合わせて、配信品質を選ぶ

```
1 let profile: (CGSize, Int) = switch quality {  
2 case .high:    (.init(width: 720, height: 1280), 2_500_000)  
3 case .medium:  (.init(width: 720, height: 1280), 1_500_000)  
4 case .low:     (.init(width: 480, height: 854),   900_000)  
5 }
```

NWPath

constrained /
expensive

presign

API latency

probe PUT

upload throughput

2.0 sec

segment target

カメラ構成前に、録音用AudioSessionを有効化

```
1 let audio = AVAudioSession.sharedInstance()
2 try audio.setCategory(
3     .playAndRecord,
4     mode: .videoRecording,
5     options: [.defaultToSpeaker, .allowBluetoothHFP]
6 )
7 try audio.setActive(true)
```

Capture

camera + microphone

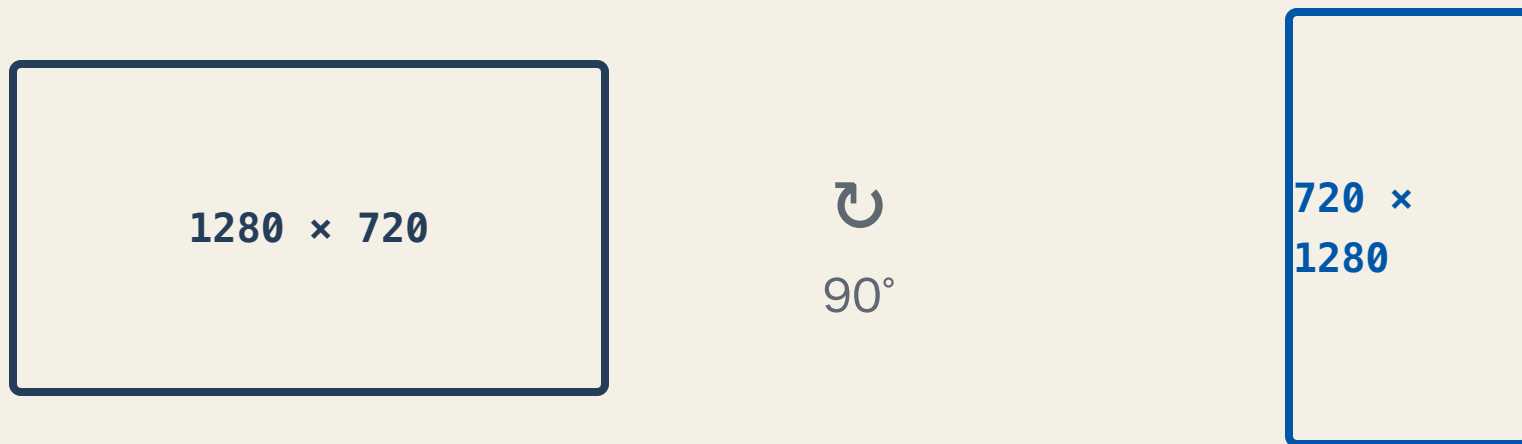
Route

speaker + Bluetooth HFP

AVCaptureSessionだけでなく、AVAudioSessionのrouteも配信品質に影響する

— ORIENTATION

縦向きは、video connectionへ90度を指定



```
1 if let connection = videoOutput.connection(with: .video),  
2   connection.isVideoRotationAngleSupported(90) {  
3     connection.videoRotationAngle = 90  
4 }
```

— COPY TIMING

映像・音声は変えず、時刻情報だけを補正

```
1 let timingInfos = try sampleTimingInfos().map { info in
2     var adjusted = info
3     adjusted.presentationTimeStamp =
4         info.presentationTimeStamp + offset
5     if info.decodeTimeStamp.isValid {
6         adjusted.decodeTimeStamp = info.decodeTimeStamp + offset
7     }
8     return adjusted
9 }
10 let copied = try CMSampleBuffer(copying: self, withNewTiming: timingInfos)
```

PTSと、有効なDTSを同じ量だけ動かす

— STOP IN THE SAME CLOCK

開始と終了を、同じ補正後timelineで閉じる

lastAdjustedPTS

最後にappendした時刻

→

endSession

media rangeを閉じる

→

markAsFinished

video / audio

→

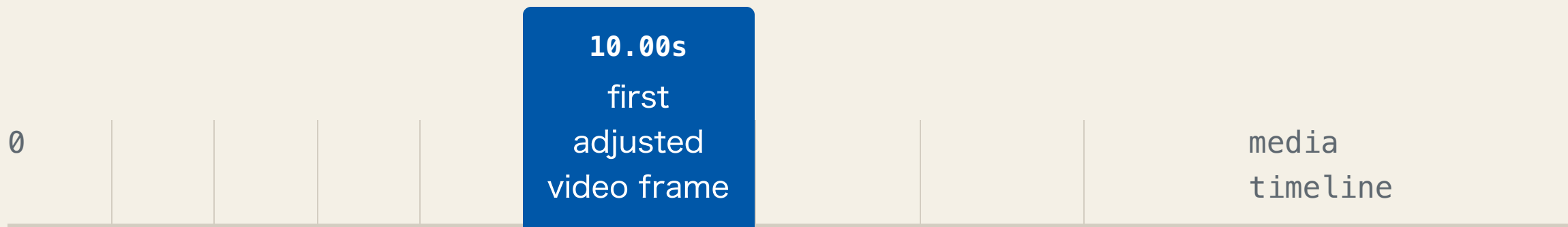
finishWriting

最後のsegmentをflush

```
1 if lastAdjustedPTS.isValid {  
2     writer.endSession(atSourceTime: lastAdjustedPTS)  
3 }
```

— INITIAL START TIME

Writerの最初のsegmentは、10秒起点に固定



```
1 private let startTimeOffset = CMTime(value: 10, timescale: 1)
2 writer.initialSegmentStartTime = startTimeOffset
```

— VIDEO SETTINGS

配信用videoフォーマットは、互換性と上り帯域を優先する

KEY	VALUE	INTENT
AVVideoCodecKey	H.264	広い再生互換性
Width / Height	480p / 720p	quality別を選択
AverageBitRate	0.9–2.5 Mbps	上り回線へ追従
ProfileLevel	High Auto	encoder profile
FrameReordering	false	decode順を単純化

LiveSegmentRecorder.makeHLSVideoSettings()

— AUDIO SETTINGS

AudioはAAC mono、品質別に48-96kbps

FORMAT	CHANNEL	SAMPLE RATE	BITRATE
AAC	mono	44.1 kHz	48-96 kbps

```
1  [  
2    AVFormatIDKey: kAudioFormatMPEG4AAC,  
3    AVNumberOfChannelsKey: 1,  
4    AVSampleRateKey: 44_100,  
5    AVEncoderBitRateKey: config.audioBitrate  
6  ]
```

実測durationを使い、取れなければ2秒

REPORT

video track duration

EXTINFへ実際の長さ。とくに最後のsegmentは2秒にならないので必要

FALLBACK

config.segmentSeconds

無効・未取得なら2.0

```
1 let duration = report?.trackReports
2   .first { $0.mediaType == .video }?
3   .duration.seconds
4
5 guard let duration, duration.isFinite, duration > 0 else {
6   return config.segmentSeconds
7 }
8 return duration
```

公開サンプルは、同じobjectを 最大3回まで再送する



segment PUTが失敗したら、そのsegmentをplaylistへ載せない

PUT中のファイルを、Viewerへ見せない

1 temporary file

request bodyを書き込む



2 flush + fsync

長さまで書き切る



3 os.replace

完成品へ一気に置換

playlistの途中状態や、半分だけのm4sをGETさせない

cache方針は、playlistとsegmentで分ける

OBJECT	CACHE	HTTP
playlist.m3u8	no-store / no-cache	毎回最新を取得
init.mp4 / m4s	max-age=31536000	Range / 206対応

S3 / CloudFrontでの設定